Linked List					
Balanced Tree (e.g. BST)	yes	no	no	yes	no
Heap (see chapter 3)	no [£]	yes	key-priority	no	yes
Hash Table	no	yes	key-value	yes [$]	no

(§) Arrays are natively static in most languages, but dynamic arrays can be built from static ones with little performance overhead.

($) Hash tables are dynamic in size when conflict resolution is resolved with chaining.

(£) Heaps only define a partial order between their keys. They allow sorting keys based on their priority, but they don’t keep any info on the order of insertion.

As part of the comparison, we also have to take into account their relative performance. But what does performance really mean for a whole data structure, if we have only discussed the running time for their individual methods?

Usually the performance for a data structure, and even more for its implementation, is a trade-off between the individual performances of its methods. This is valid for all data structures: there is no ideal data structure that has optimal performance for all the operations it provides. For instance, arrays are faster for random, position-based access. But they are slow when their shape needs to be changed, and very slow when you need to look up for an element by value. Lists allow for fast insertion of new elements, but lookup and access by position are slow. Hash tables, which we will discuss later, have fast key-based lookup, but finding the successor of an element, or maximum and minimum, are very slow.

In practice, choosing the best data structure should be based on a careful analysis of your problem and the performance of the data structure (DT), but it's more important to avoid using a bad data structure (which could cause a bottleneck) than to find the single best option available, just to gain a small saving on the second choice.